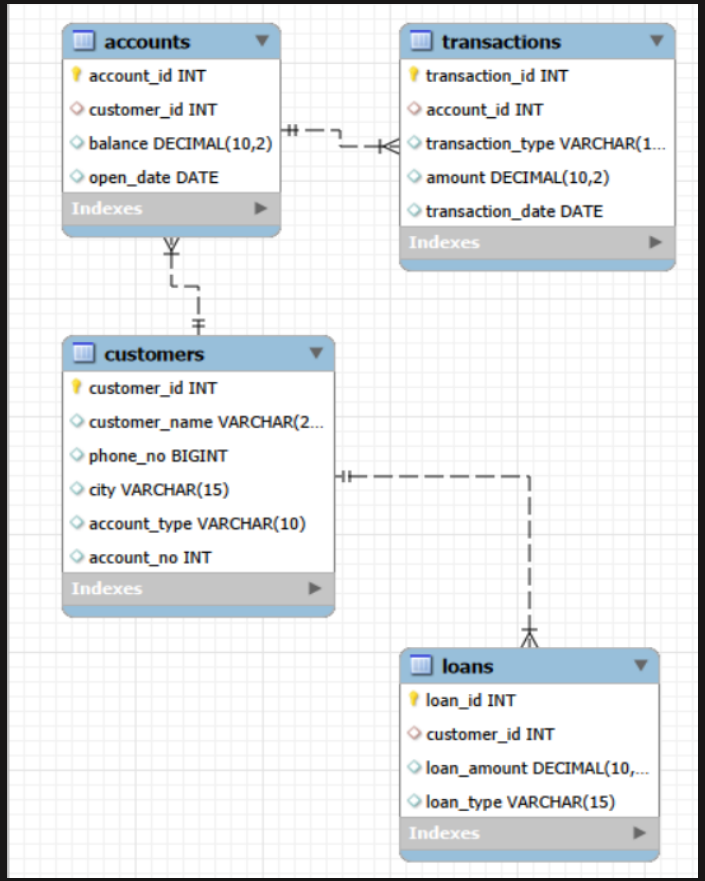
Banking Management System

Professional Banking Management System developed by **Shriharsh Shinde** — demonstrating database architecture, customer and transaction management, loan processing, financial data analysis, and advanced SQL implementation through joins, aggregations, subqueries, and analytical window functions within a modern banking environment.

ER Diagram :-


accounts
account_id INT
customer_id INT
balance DECIMAL(10,2)
open_date DATE
Indexes
transactions
transaction_id INT
account_id INT
transaction_type VARCHAR(1...
amount DECIMAL(10,2)
transaction_date DATE
Indexes
customers
customer_id INT
customer_name VARCHAR(2...
phone_no BIGINT
city VARCHAR(15)
account_type VARCHAR(10)
account_no INT
Indexes
loans
loan_id INT
customer_id INT
loan_amount DECIMAL(10,...
loan_type VARCHAR(15)
Indexes

1.Display customer names, account numbers, and account balances using INNER JOIN

```
select c.customer_name,a.balance
from customers c inner join accounts a
on c.customer_id=a.customer_id;
```

customer_name	balance
Rahul Sharma	55000.00
Sneha Patil	120000.00
Aman Verma	35000.00
Priya Singh	98000.00
Karan Mehta	75000.00
Neha Joshi	150000.00
Rohit Kumar	42000.00
Pooja Sharma	88000.00
Vivek Shah	200000.00
Anjali Verma	67000.00

2. Find the top 3 customers with the highest account balances.

```sql
select c.customer_name,a.balance
from customers c inner join accounts a
on c.customer_id=a.customer_id
order by a.balance asc
limit 3;
```

customer_name	balance
Aman Verma	35000.00
Rohit Kumar	42000.00
Rahul Sharma	55000.00

3. Show all customers who have taken loans along with loan amount and loan type.

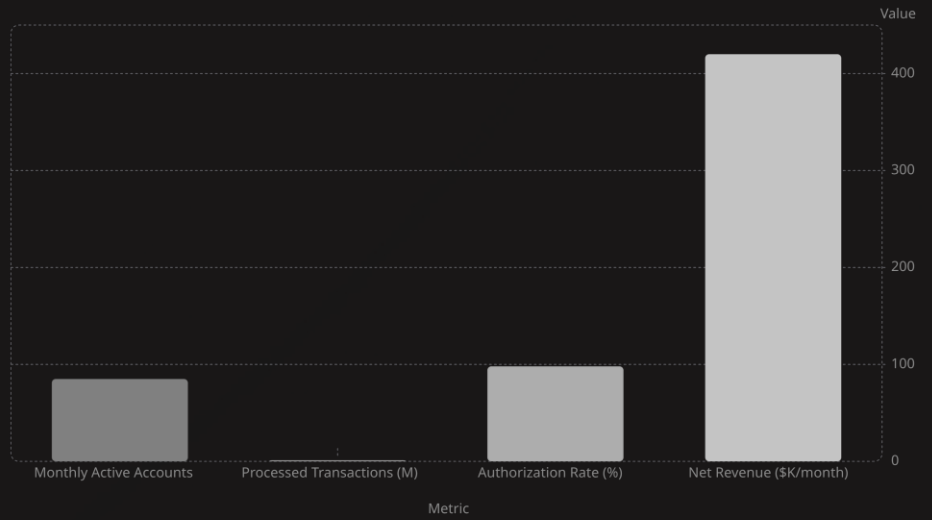
```
select c.customer_name,l.loan_amount,l.loan_type
from customers c  right outer join loans l
on c.customer_id=l.customer_id;
```

customer_name	loan_amount	loan_type
Rahul Sharma	500000.00	Home Loan
Sneha Patil	200000.00	Car Loan
Priya Singh	100000.00	Education Loan
Karan Mehta	300000.00	Business Loan
Neha Joshi	150000.00	Personal Loan
Pooja Sharma	250000.00	Home Loan
Vivek Shah	400000.00	Business Loan
Anjali Verma	180000.00	Car Loan

4. Find the total deposited amount and total withdrawn amount separately.

```sql
select transaction_type,sum(amount) as total
from transactions
group by transaction_type;
```

	transaction_type	total
▶	Deposit	121000.00
	Withdraw	35000.00



5. Display customer-wise total transaction amount using GROUP BY.

```sql
select c.customer_name,sum(t.amount) as total_transaction
from customers c inner join accounts a
on c.customer_id = a.customer_id
inner join transactions t
on a.account_id = t.account_id
group by c.customer_name;
```

customer_name	total_transaction
Rahul Sharma	15000.00
Sneha Patil	35000.00
Aman Verma	8000.00
Priya Singh	15000.00
Karan Mehta	15500.00
Neha Joshi	7000.00
Rohit Kumar	24000.00
Pooja Sharma	4500.00
Vivek Shah	30000.00
Anjali Verma	2000.00

6. Find customers whose balances are greater than the average bank balance.

```sql
select c.customer_name, a.balance
from customers c
inner join accounts a
on c.customer_id = a.customer_id
where a.balance > (
select avg(balance)
from accounts
);
```

	customer_name	balance
▶	Sneha Patil	120000.00
	Priya Singh	98000.00
	Neha Joshi	150000.00
	Vivek Shah	200000.00

7. Show the highest transaction amount performed by each customer.

```sql
select c.customer_name,sum(t.amount) as total_amount
from customers c inner join accounts a
on c.customer_id=a.customer_id
inner join transactions t
on a.account_id=t.account_id
group by c.customer_name;
```

customer_name	total_amount
Rahul Sharma	15000.00
Sneha Patil	35000.00
Aman Verma	8000.00
Priya Singh	15000.00
Karan Mehta	15500.00
Neha Joshi	7000.00
Rohit Kumar	24000.00
Pooja Sharma	4500.00
Vivek Shah	30000.00
Anjali Verma	2000.00

8. Display all customers who have not taken any loans using LEFT JOIN.

```sql
select c.customer_name
from customers c left join loans l
on c.customer_id = l.customer_id
where l.loan_id is null;
```

customer_name
Aman Verma
Rohit Kumar

9. Find the total number of transactions performed by each customer.

```
select c.customer_name,sum(t.amount) as total_amount
from customers c inner join accounts a
on c.customer_id=a.customer_id
inner join transactions t
on a.account_id=t.account_id
group by c.customer_name;
```

customer_name	total_amount
Rahul Sharma	15000.00
Sneha Patil	35000.00
Aman Verma	8000.00
Priya Singh	15000.00
Karan Mehta	15500.00
Neha Joshi	7000.00
Rohit Kumar	24000.00
Pooja Sharma	4500.00
Vivek Shah	30000.00
Anjali Verma	2000.00

10. Rank customers based on their account balances using RANK() window function.

```
select c.customer_name,a.balance,rank() over (order by a.balance desc) as balance_rank
from customers c inner join accounts a
on c.customer_id=a.customer_id;
```

customer_name	balance	balance_rank
Vivek Shah	200000.00	1
Neha Joshi	150000.00	2
Sneha Patil	120000.00	3
Priya Singh	98000.00	4
Pooja Sharma	88000.00	5
Karan Mehta	75000.00	6
Anjali Verma	67000.00	7
Rahul Sharma	55000.00	8
Rohit Kumar	42000.00	9
Aman Verma	35000.00	10

11. Display dense ranking of customers according to balance using DENSE_RANK().

```
select c.customer_name,a.balance,dense_rank() over (order by a.balance desc) as balance_rank
from customers c inner join accounts a
on c.customer_id=a.customer_id;
```

customer_name	balance	balance_rank
Vivek Shah	200000.00	1
Neha Joshi	150000.00	2
Sneha Patil	120000.00	3
Priya Singh	98000.00	4
Pooja Sharma	88000.00	5
Karan Mehta	75000.00	6
Anjali Verma	67000.00	7
Rahul Sharma	55000.00	8
Rohit Kumar	42000.00	9
Aman Verma	35000.00	10

12. Show previous transaction amount using LAG() function

```
select amount,lag(amount) over (order by amount asc) as previous_amount
from transactions;
```

amount	previous_amount
2000.00	NULL
3000.00	2000.00
3500.00	3000.00
4500.00	3500.00
5000.00	4500.00
5000.00	5000.00
7000.00	5000.00
9000.00	7000.00
10000.00	9000.00
10000.00	10000.00
12000.00	10000.00
15000.00	12000.00
15000.00	15000.00
25000.00	15000.00
30000.00	25000.00

13. Show next transaction amount using LEAD() function.

```
select amount,lead(amount) over (order by amount asc) as next_amount
from transactions;
```

amount	next_amount
2000.00	3000.00
3000.00	3500.00
3500.00	4500.00
4500.00	5000.00
5000.00	5000.00
5000.00	7000.00
7000.00	9000.00
9000.00	10000.00
10000.00	10000.00
10000.00	12000.00
12000.00	15000.00
15000.00	15000.00
15000.00	25000.00
25000.00	30000.00
30000.00	NULL

14. Calculate running total of transaction amounts using SUM() OVER().

```
select transaction_id,amount,sum(amount) over (order by transaction_date) as running_total
from transactions;
```

transaction_id	amount	running_total
1	10000.00	10000.00
2	5000.00	40000.00
3	25000.00	40000.00
4	3000.00	43000.00
5	15000.00	58000.00
6	12000.00	77000.00
7	7000.00	77000.00
8	9000.00	90500.00
9	4500.00	90500.00
10	30000.00	122500.00
11	2000.00	122500.00
12	10000.00	137500.00
13	5000.00	137500.00
14	3500.00	156000.00
15	15000.00	156000.00

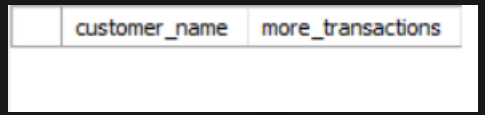
15. Find the second highest account balance using subquery or window function.

```sql
select max(balance) as highest_balance
from accounts
where balance <(
select max(balance)
from accounts
);
```

	highest_balance
▶	150000.00

16. Find customers who performed more than 2 transactions.

```
select c.customer_name,count(t.transaction_id) as more_transactions
from customers c inner join accounts a
on c.customer_id=a.customer_id
inner join transactions t
on a.account_id=t.account_id
group by c.customer_name
having count(t.transaction_id)>2;
```



	customer_name	more_transactions

17. Display customer-wise minimum and maximum transaction amounts

```
select c.customer_name,max(t.amount) as maximum_amounts ,min(t.amount) as minimum_amounts
from customers c inner join accounts a
on c.customer_id=a.customer_id
inner join transactions t
on a.account_id=t.account_id
group by c.customer_name;
```

customer_name	maximum_amounts	minimum_amounts
Rahul Sharma	10000.00	5000.00
Sneha Patil	25000.00	10000.00
Aman Verma	5000.00	3000.00
Priya Singh	15000.00	15000.00
Karan Mehta	12000.00	3500.00
Neha Joshi	7000.00	7000.00
Rohit Kumar	15000.00	9000.00
Pooja Sharma	4500.00	4500.00
Vivek Shah	30000.00	30000.00
Anjali Verma	2000.00	2000.00

Thank you

- Email:- shriharshshinde8@gmail.com
- LinkedIn:-https://www.linkedin.com/in/shriharsh-shinde-3b0530328